DESENVOLVIMENTO MOBILE

AULA 6 — GALERIA, CÂMERA E TELAS DE VERDADE

expo-image-picker · Expo Router · CESAR School 2026.2

AGENDA

1. `expo-image-picker` : a galeria
2. `expo-image-picker` : a câmera e as permissões
3. 🖐️ Checkpoint 1 · ☕ Intervalo
4. **Expo Router**: roteamento por arquivos
5. `_layout.tsx` e `Stack` : a pilha de telas
6. `Link` e `router` : andar pela pilha
7. 🖐️ Checkpoint 2
8. Levando para o seu projeto
9. Hands-on guiado · Atividade aplicada

AO FINAL DA AULA VOCÊ DEVE CONSEGUIR

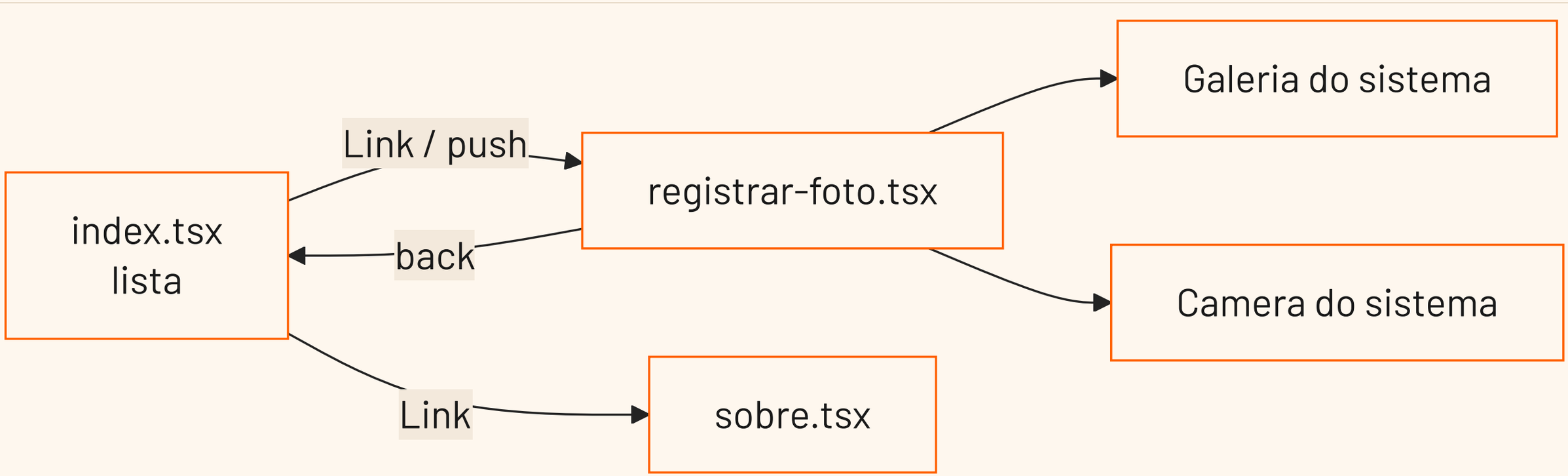
1. Escolher foto da galeria e ler `{ canceled, assets }` corretamente
2. Tirar foto com a câmera do sistema tratando os **três estados** da permissão
3. Decidir entre `expo-image-picker` e `expo-camera`
4. Prever o **endereço** de uma tela só olhando o caminho do arquivo
5. Montar o `_layout.tsx` com `Stack` e dar **título** a cada tela
6. Navegar com `Link` e com `router.push` / `router.back` — e saber **qual** usar
7. Explicar a navegação como uma **pilha**
8. Migrar um projeto de `App.tsx` para o Expo Router

O QUE ENTRA HOJE

- `expo-image-picker` — galeria e câmera do sistema
- **A base do Expo Router** — arquivos viram telas, `_layout` com `Stack`, `Link` e `router`

> Hoje cada tela **se vira sozinha**. Levar informação de uma tela para outra é o assunto da **próxima aula**.

DUAS FERRAMENTAS, UM APP COM CARA DE APP



FOTOS QUE O USUÁRIO JÁ TEM E TELAS COM ENDEREÇO.

PARTE 1

`expo-image-picker`: A GALERIA

COZINHAR × PEDIR NO BALCÃO

`expo-image-picker` abre a **tela do sistema** para o usuário escolher uma foto da galeria ou tirar uma com a câmera — e **devolve o arquivo** para o seu app.

- 🍳 **`expo-camera` (Aula 5) é montar a cozinha:** fogão, panela, botão, layout — tudo seu
- 🛎️ **`expo-image-picker` é pedir no balcão:** o sistema entrega o prato pronto, do jeito dele

O QUE ISSO SIGNIFICA NA PRÁTICA

	`expo-camera` (Aula 5)	`expo-image-picker` (hoje)
A interface	**sua** — `CameraView` na tela	**do sistema** — abre por cima
Galeria	não tem	tem
Recorte	você construiria	`allowsEditing`
Flash, zoom, sobreposição	controle total	nenhum
Código	gate + `CameraView` + referência	**uma chamada** `await`

> Se a câmera **é** a funcionalidade → `expo-camera`. Se a foto é **um anexo** → `expo-image-picker`.

O CONTRATO DE TRÊS TEMPOS, DE NOVO

```
npx expo install expo-image-picker
```

Pacote	Pedir	Ler	Parar
picker — galeria	**não precisa**	`launchImageLibraryAsync()`	—
picker — câmera	`requestCameraPermissionsAsync()`	`launchCameraAsync()`	—

É UM COPO D'ÁGUA, NÃO UMA TORNEIRA: NÃO EXISTE `remove()`.

POR QUE A GALERIA NÃO PEDE PERMISSÃO

- Quem mostra as fotos é o **sistema**, não o seu app
- O seu app só recebe a foto que o usuário **escolheu**
- O usuário já decidiu — **foto por foto**

A PERMISSÃO ACOMPANHA O RISCO (AULA 5).

ABRIR A GALERIA

```
import * as ImagePicker from 'expo-image-picker';

async function escolherDaGaleria() {
  const resultado = await ImagePicker.launchImageLibraryAsync({
    mediaTypes: ['images'],
    allowsEditing: true,
    aspect: [4, 3],
    quality: 0.7,
  });

  if (resultado.canceled) return;
  setFotoUri(resultado.assets[0].uri);
}
```

AS OPÇÕES

Opção	Valores	Default	O que faz
`mediaTypes`	`['images']`	`'images'`	o que aparece para escolher
`allowsEditing`	booleano	`false`	abre o **recorte** depois da escolha
`aspect`	`[largura, altura]`	—	proporção do recorte — **só Android**
`quality`	**0 a 1**	`1`	compressão do JPEG
`allowsMultipleSelection`	booleano	`false`	escolher várias
`selectionLimit`	número	`0` = máximo	teto da seleção
`base64`	booleano	`false`	**não peça sem precisar**

O RESULTADO: `canceled` E `assets`

```
// usuário escolheu
{ canceled: false, assets: [{ uri, width, height, fileName, fileSize, mimeType, type }] }
// usuário desistiu
{ canceled: true, assets: null }
```

- **Cancelar não é erro.** Não cai em `catch`. É resultado **normal**
- `assets` **é sempre uma lista** — mesmo com uma foto só. Por isso `assets[0]`
- Depois do `if (resultado.canceled) return;`, o TypeScript **sabe** que `assets` não é `null`

ZUMBIS DO PICKER

O tutorial que você vai achar	O que compila hoje
`mediaTypes: ImagePicker.MediaTypeOptions.Images`	`mediaTypes: ['images']` — **deprecado**
`if (!result.cancelled)` — dois "l"	**`result.canceled`** — um "l"
`setImage(result.uri)`	**`result.assets[0].uri`**
`Permissions.askAsync(Permissions.CAMERA_ROLL)`	cada pacote traz as suas funções
`requestCameraRollPermissionsAsync()`	`requestMediaLibraryPermissionsAsync()`

O PIOR DOS CINCO É O `cancelled`.

EM JAVASCRIPT SEM TIPAGEM, `result.cancelled` É `undefined` — FALSO — E O CÓDIGO SEGUE COMO SE O USUÁRIO TIVESSE ESCOLHIDO. O TYPESCRIPT PEGA; O TUTORIAL EM JS, NÃO.

RECORTE: CADA PLATAFORMA DO SEU JEITO

	Android	iOS
`allowsEditing`	recorta **e gira**	só recorta
`aspect: [4, 3]`	respeitado	**ignorado** — recorte sempre **quadrado**

A OPÇÃO EXISTE, COMPILA, E FAZ COISAS DIFERENTES DE CADA LADO.

Mesma lição do `timeInterval` da Aula 5: **declare** — não prometa "4:3" a quem usa iPhone.

SELEÇÃO MÚLTIPLA

```
const resultado = await ImagePicker.launchImageLibraryAsync({
  mediaTypes: ['images'],
  allowsMultipleSelection: true,
  selectionLimit: 5,
});
if (!resultado.canceled) setFotos(resultado.assets);
```

- `allowsMultipleSelection` **desliga** o `allowsEditing` — não dá para recortar cinco de uma vez
- O resultado já é uma **lista** → direto para uma `FlatList` (Aula 4)

EXIBIR — E ONDE ESTÁ O ARQUIVO

```
import { Image } from 'expo-image';

<Image source={{ uri: fotoUri }} style={estilos.previa} contentFit="cover" transition={300} />
```

- **Nada novo:** o `expo-image` da Aula 5 — `contentFit` explícito, **import conferido**
- O `uri` aponta para uma **cópia no cache do app**
- Igual à foto do `takePictureAsync`: **não é permanente**

PARTE 2

A CÂMERA E AS PERMISSÕES

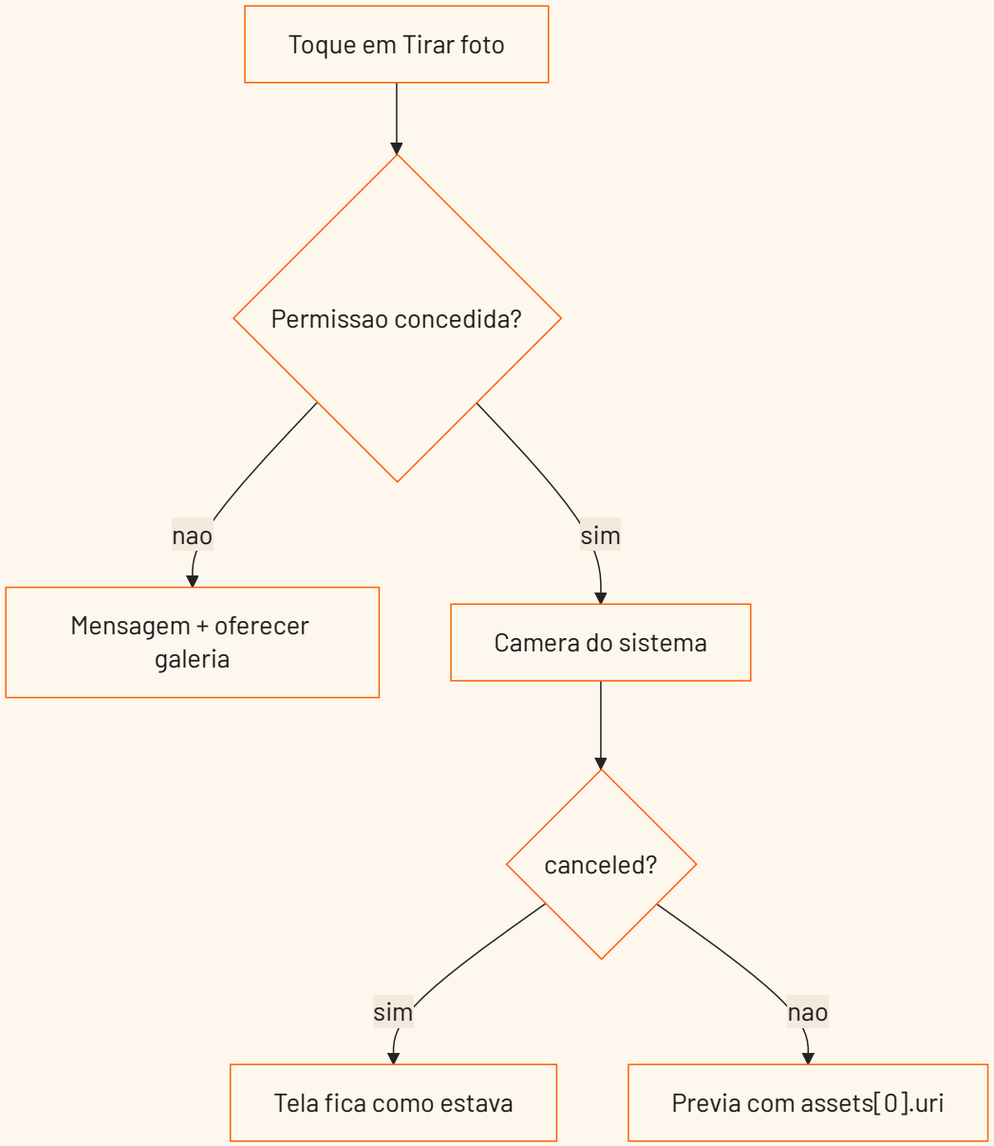
`launchCameraAsync` : MESMA CHAMADA, OUTRA FONTE

```
async function tirarFoto() {
  const permissao = await ImagePicker.requestCameraPermissionsAsync();
  if (!permissao.granted) {
    setFalha(permissao.canAskAgain ? 'camera-negada' : 'camera-bloqueada');
    return;
  }

  const resultado = await ImagePicker.launchCameraAsync({
    mediaTypes: ['images'],
    allowsEditing: true,
    aspect: [4, 3],
    quality: 0.7,
  });

  if (resultado.canceled) return;
  setFotoUri(resultado.assets[0].uri);
}
```

O CAMINHO DA FOTO



OS TRÊS ESTADOS DA PERMISSÃO

Estado	O que a tela faz
`granted`	abre a câmera
`!granted` e `canAskAgain: true`	explica, deixa pedir de novo — e **oferece a galeria**
`!granted` e `canAskAgain: false`	o diálogo **não volta**: explica o caminho nas **Configurações** do aparelho

> **Sempre existe um caminho:** a galeria não precisa de permissão.

AS DUAS FORMAS DE PERGUNTAR

Forma	Câmera	Galeria
chamada direta, no `onPress`	`requestCameraPermissionsAsync()`	`requestMediaLibraryPermissionsAsync()`
o par `[estado, pedir]`, no topo	`useCameraPermissions()`	`useMediaLibraryPermissions()`

PARA O PICKER, A CHAMADA DIRETA É A NATURAL.

A permissão só importa **no momento do toque** — a tela não muda de forma por causa dela.

⚠️ A DOCUMENTAÇÃO DISCORDA DELA MESMA

- O **texto** do `expo-image-picker`: *"No permissions request is necessary for launching the image library."*
- O **exemplo oficial**, logo abaixo: pede `requestMediaLibraryPermissionsAsync()` antes

AS DUAS COISAS SÃO VERDADE EM CONTEXTOS DIFERENTES.

- o picker **do sistema** não precisa
- a permissão de biblioteca é para quem vai **ler a galeria inteira** por conta própria

> A aula segue o texto. E a lição é maior que a regra: **leia documentação com desconfiança**.

O CONFIG PLUGIN

```
{
  "expo": {
    "plugins": [
      [
        "expo-image-picker",
        {
          "photosPermission": "Usamos suas fotos para você anexar a imagem do seu hábito.",
          "cameraPermission": "Usamos a câmera para você registrar a foto do seu hábito.",
          "microphonePermission": false
        }
      ]
    ]
  }
}
```

- Escreva o **porquê** — o usuário lê antes de decidir
- `microphonePermission: false` : por padrão o pacote pede microfone no Android (por causa de vídeo)

SIMULADOR, EMULADOR E APARELHO

Onde	Galeria	Câmera
iOS Simulator	✅ — arraste imagens do Mac para dentro	❌ **não tem câmera**
Android Emulator	✅	✅ câmera virtual (cena 3D)
Aparelho físico	✅	✅

GALERIA EM QUALQUER LUGAR; CÂMERA EM APARELHO FÍSICO.

CITADO, NÃO ENSINADO

Recurso	O que é	Por que não hoje
`mediaTypes: ['videos']`	escolher ou gravar vídeo	outro arquivo, outra permissão (microfone)
`getPendingResultAsync()`	recupera a foto quando o Android **mata o app** com a câmera aberta	caso raro, de aparelho com pouca memória
salvar na galeria	outro pacote	não está no programa

✋ CHECKPOINT 1 — PICKER

1. Foto de perfil: `expo-camera` ou `expo-image-picker`? E um leitor de QR code?
2. Por que a galeria não pede permissão e a câmera pede?
3. O usuário tocou em "Cancelar". O que vem em `resultado`? Cai no `catch`?
4. `if (!result.cancelled) setImage(result.uri)` — quantos problemas?
5. Pedi `aspect: [4, 3]` e no iPhone veio quadrado. É bug?

☕ INTERVALO — 10 MIN

PARTE 3

ROTEAMENTO POR ARQUIVOS: A PASTA É O MAPA

NA AULA PASSADA, O VOLTAR DO ANDROID **FECHOU O APP INTEIRO**.

HOJE O APP GANHA TELAS DE VERDADE — COM ENDEREÇO, PILHA E UM VOLTAR QUE VOLTA.

UM APP DE VERDADE TEM VÁRIAS TELAS

- Até agora: **um componente só**, que trocava o que mostrava com um `if`
- Um app de verdade: o usuário **anda** — entra, volta, entra em outra
- Esse andar tem regras que o sistema **já conhece**:
  - gesto de voltar no iPhone
  - botão de voltar no Android
  - título no topo de cada tela

VOCÊ NÃO QUER REIMPLEMENTAR NENHUMA DELAS.

O QUE É O EXPO ROUTER

Cada arquivo dentro de `src/app/` vira uma tela — e o caminho do arquivo **é** o endereço da tela.

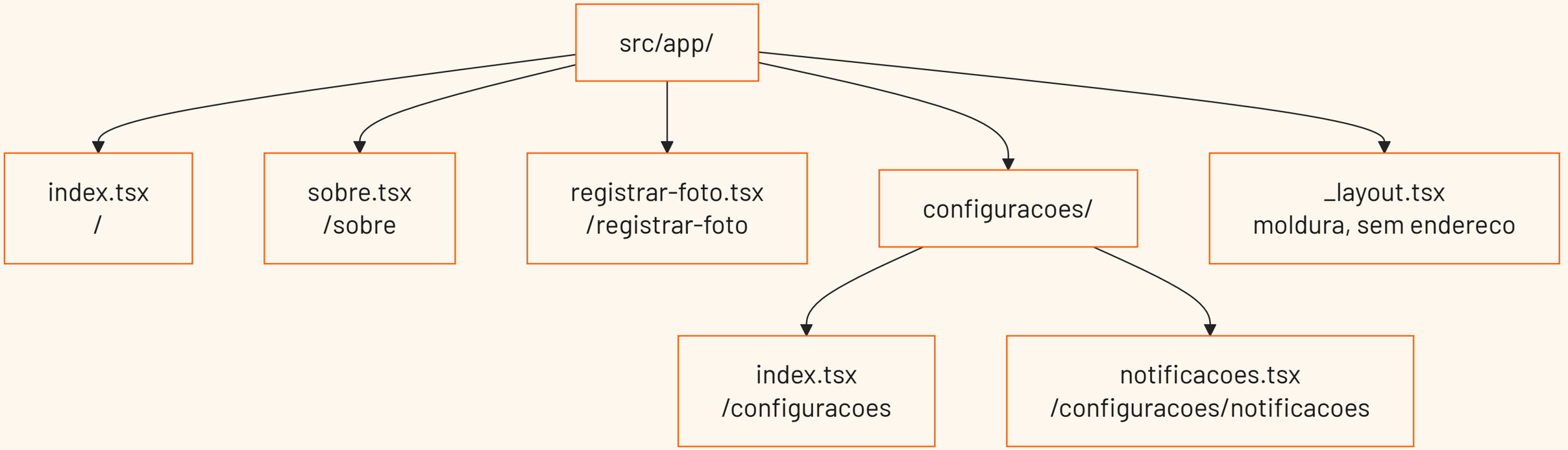
- 🏢 **O prédio e a placa da porta:** a pasta é a planta; cada arquivo é uma sala; o nome está na porta
- Para mandar alguém a uma sala, você não descreve o caminho — diz o número: `/sobre`
- Quem já fez web: é o `sobre.html` que vira `/sobre`. E o mesmo app roda na web

VOCÊS JÁ USAM O ROUTER DESDE A AULA 3 — SEM SABER.

A REGRA DE CONVERSÃO

Arquivo	Endereço (`href`)
`src/app/index.tsx`	`/`
`src/app/sobre.tsx`	`/sobre`
`src/app/registrar-foto.tsx`	`/registrar-foto`
`src/app/configuracoes/index.tsx`	`/configuracoes`
`src/app/configuracoes/notificacoes.tsx`	`/configuracoes/notificacoes`
`src/app/_layout.tsx`	**nenhum** — é moldura, não tela

A PASTA, DESENHADA



TRÊS REGRAS QUE SAEM DA TABELA

1. **`index.tsx` é a tela padrão da pasta** — em `src/app/`, é a primeira tela do app
2. **Pasta vira pedaço do endereço** — subpasta = mais um `/`
3. **Arquivo que começa com `_` não é tela** — o `_layout.tsx`

> Nome de arquivo vira URL: **minúsculas e hífen** — `registrar-foto.tsx`, nunca `RegistrarFoto.tsx`.

UMA TELA É SÓ UM COMPONENTE NUM LUGAR ESPECIAL

```tsx
// src/app/sobre.tsx
import { StyleSheet, Text, View } from 'react-native';

export default function TelaSobre() {
  return (
    <View style={estilos.tela}>
      <Text>Rastreador de Micro-hábitos — versão da Aula 6.</Text>
    </View>
  );
}

const estilos = StyleSheet.create({
  tela: { flex: 1, padding: 24, justifyContent: 'center' },
});
```

O ERRO Nº 1: COMPONENTE DENTRO DE `src/app/`

- **Tudo** em `src/app/` é tratado como rota
- `CardHabito.tsx` lá dentro vira uma tela `/CardHabito` — que ninguém pediu
- O resto mora **fora**: `src/components/`, `src/types/`, `src/services/`, `src/screens/`

> O próprio template avisa, no `npm run reset-project`: *"Put all your application code in /src, only screens and layout files should be in /src/app."*

ZUMBI: `app/` × `src/app/`

O que valia no passado	O que vale hoje
As rotas ficavam em `app/`, na raiz do projeto	Desde o **SDK 55**, o template cria `src/app/`
—	Se existirem as duas, **só `src/app/` é usada**

- `app/` na raiz **ainda funciona** — mas não é mais recomendado!
- **Não misture:** com as duas pastas, a de fora é **ignorada em silêncio**

LEIA O TERMINAL: `Using src/app as the root directory for Expo Router.`

QUEM LIGA O ROUTER

Nas aulas anteriores: `index.ts` com `registerRootComponent(App)`.

Com o Router, quem começa o app é o próprio Router:

```
{ "main": "expo-router/entry" }
```

NÃO EXISTE MAIS `App.tsx`. O PAPEL DELE FOI DIVIDIDO EM DOIS:

- a **moldura** → `src/app/_layout.tsx`
- a **primeira tela** → `src/app/index.tsx`

PARTE 4

`_layout.tsx` E `Stack`: A PILHA DE TELAS

O QUE É UM LAYOUT

O `_layout.tsx` **envolve** as telas da pasta e decide **como** se anda entre elas.

- 🖼️ **A moldura do quadro:** as telas são os quadros que você troca; o layout é a moldura que fica
- O cabeçalho com título e voltar **é da moldura**, não do quadro
- Hoje a moldura é sempre um **`Stack`** — pilha

O LAYOUT MÍNIMO

```tsx
// src/app/_layout.tsx
import { Stack } from 'expo-router';

export default function LayoutRaiz() {
  return <Stack />;
}
```

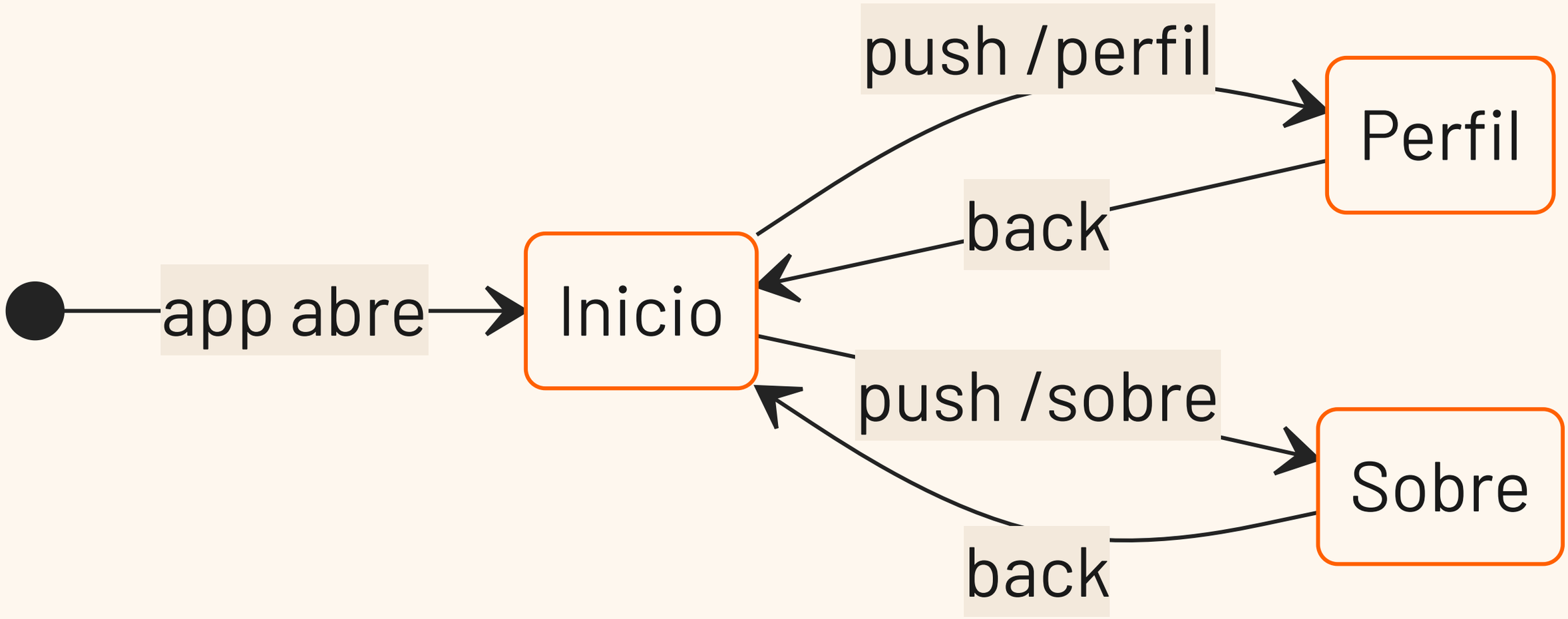
É EXATAMENTE O ARQUIVO QUE O `reset-project` GERA.

Só com isso: cabeçalho, botão de voltar, gesto do iOS e voltar do Android.

A PILHA DE PRATOS

CADA TELA NOVA É UM PRATO COLOCADO EM CIMA (`push`). VOLTAR É TIRAR O DE CIMA (`back`).

A PILHA, DESENHADA



"NO MESMO ESTADO EM QUE FICOU"

A TELA DE BAIXO

- **não** é recriada quando você volta
- rolou a lista até o meio? continua no meio

A TELA DE CIMA

- sai da pilha e é **destruída**
- o `useState` dela **some**

> Escolheu uma foto em `/registrar-foto` e voltou? **A foto se foi.** Guardem isso.

CONFIGURAR AS TELAS

```tsx
// src/app/_layout.tsx
import { Stack } from 'expo-router';

export default function LayoutRaiz() {
  return (
    <Stack screenOptions={{ headerTintColor: '#f26522' }}>
      <Stack.Screen name="index" options={{ title: 'Meus hábitos' }} />
      <Stack.Screen name="registrar-foto" options={{ title: 'Foto do hábito' }} />
      <Stack.Screen name="sobre" options={{ title: 'Sobre o app' }} />
    </Stack>
  );
}
```

AS TRÊS REGRAS DO `Stack.Screen`

1. `name` **é o nome do arquivo sem extensão** — `name="registrar-foto"` → `registrar-foto.tsx`
2. `screenOptions` **vale para todas;** `options` **vale para uma** — a de uma tela vence
3. **Listar não é obrigatório** — arquivo já é rota. Você lista para **configurar**, não para **registrar**

EXPO NAVIGATION - STACK

Opção	Para quê
`title`	texto do cabeçalho
`headerShown`	`false` esconde o cabeçalho
`headerTintColor`	cor do título e do voltar
`headerStyle`	`{ backgroundColor }` do cabeçalho
`headerBackTitle`	texto ao lado da seta, no iOS

PARTE 5

`Link` E `router`: ANDAR PELA PILHA

`Link` : O CAMINHO DECLARADO

```tsx
import { Link } from 'expo-router';

<Link href="/sobre" style={estilos.link}>
  Sobre o app
</Link>

<Link href="/registrar-foto" asChild>
  <Pressable style={estilos.botao}>
    <Text style={estilos.textoBotao}>Registrar com foto</Text>
  </Pressable>
</Link>
```

- `Link` é um `Text` **que navega**
- **`asChild`** : o `Link` passa o toque para o **seu** `Pressable` (Aula 4)

`href` COMEÇA COM `/`

- `href="/sobre"` → endereço **absoluto**
- `href="sobre"` → endereço **relativo** à tela atual — numa subpasta, leva a **outro lugar**

UTILIZE SEMPRE COM BARRA.

`router` : NAVEGAR DEPOIS DE UMA AÇÃO

```tsx
import { useRouter } from 'expo-router';

export default function TelaInicial() {
  const router = useRouter();
  // ...
  <Pressable onPress={() => router.push('/registrar-foto')}>...</Pressable>
}
```

Função	Na pilha
`router.push('/rota')`	**empilha**
`router.back()`	**desempilha** — o mesmo que o voltar
`router.canGoBack()`	existe tela embaixo?
`router.replace('/rota')`	**troca** o topo, sem deixar a anterior *(citar)*

`Link` OU `router.push`?

A NAVEGAÇÃO ACONTECE PORQUE O USUÁRIO TOCOU NUM LINK — OU DEPOIS QUE ALGO TERMINOU?

Situação	Use
"Toque em *Sobre* leva para a tela Sobre"	**`Link`** — destino fixo, escrito na tela
"Depois que a foto foi escolhida, volta"	**`router.back()`**, depois do `await`

> **`Link` no JSX; `router` dentro de função.**

O TYPESCRIPT CONHECE AS SUAS ROTAS

O template vem com `"experiments": { "typedRoutes": true }`. Depois do primeiro `npx expo start`, o Router **gera os tipos** das rotas.

```
<Link href="/novaFoto">...</Link>   // o arquivo se chama registrar-foto.tsx
```

```
Type '"/novaFoto"' is not assignable to type ...
Did you mean '"/registrar-foto"'?
```

`href` ERRADO É PEGO NO EDITOR, ANTES DE RODAR.

DUAS PLATAFORMAS, UM VOLTAR

	Voltar
iOS	seta no cabeçalho + **gesto de arrastar da borda esquerda**
Android	seta no cabeçalho + **botão/gesto de voltar do sistema**

OS DOIS CHAMAM O MESMO `back`. VOCÊ NÃO ESCREVE NADA.

ZUMBIS DA NAVEGAÇÃO

O que valia no passado	O que vale hoje
`NavigationContainer` + `createNativeStackNavigator()` no `App.tsx`	`<Stack />` no `_layout.tsx`
`navigation.navigate('Sobre')` — **nome** da tela	`router.push('/sobre')` — **endereço**
`import ... from '@react-navigation/native'` no app	importar de **`expo-router`** (SDK 56+)
" `router.navigate` volta se a tela já existe"	desde o SDK 52, **`navigate` empilha** — use `push`
"Expo Router v4 / v5"	o pacote segue o SDK: `expo-router@57`

HOJE CADA TELA **SE VIRA SOZINHA**.

ABRIR "O HÁBITO QUE EU TOQUEI" OU LEVAR A FOTO PARA A LISTA EXIGE LEVAR INFORMAÇÃO DE UMA TELA PARA OUTRA. É A PRÓXIMA AULA.

🖐 CHECKPOINT 2 — ROTAS E PILHA

1. Qual o endereço de `src/app/configuracoes/index.tsx`? E de `src/app/_layout.tsx`?
2. Coloquei o `CardHabito.tsx` em `src/app/`. O que acontece, e onde ele deveria estar?
3. Fiz `push('/registrar-foto')` e depois `back()`. A lista da tela inicial é recriada? E o `useState` da tela de foto?
4. Botão "Sobre": `Link` ou `router.push`? E o "Concluir", que só volta **depois** da foto escolhida?
5. O tutorial diz `navigation.navigate('Sobre')`. Como fica no nosso projeto?

PARTE 6

LEVANDO PARA O SEU PROJETO: DE `App.tsx` PARA `src/app/`

A MIGRAÇÃO — PARTE 1

```
npx expo install expo-router react-native-safe-area-context react-native-screens expo-linking expo-constants expo-status-bar
npx expo install expo-image-picker
```

1. **Ponto de entrada** no `package.json`: `"main": "expo-router/entry"`
2. `scheme` no `app.json` — ex.: `"scheme": "habittracker"`
3. O `npx expo install` **já adiciona** `"expo-router"` em `plugins` — **confira, não duplique**

A MIGRAÇÃO — PARTE 2

4. **Moldura:** `src/app/_layout.tsx` com `<Stack />`
5. **A tela atual vira a rota inicial** — sem mover código:

```
// src/app/index.tsx
export { default } from '../screens/tela-habitos';
```

6. **Apagar** `App.tsx` e `index.ts`
7. **Reiniciar limpando o cache:** `npx expo start --clear`

CONFERIR: `Using src/app as the root directory for Expo Router.`

O QUE COSTUMA DAR ERRADO NA MIGRAÇÃO

Sintoma	Causa
o app abre o `App.tsx` antigo, ou reclama que ele sumiu	`"main"` não foi trocado
uma tela aparece duas vezes, ou uma rota estranha surge	componente **dentro** de `src/app/` *e* em `src/screens/`
"editei e não mudou"	faltou `--clear`, ou existe `app/` **e** `src/app/`
`"expo-router"` duas vezes em `plugins`	adicionado à mão depois do `expo install`

HANDS-ON GUIADO — 30 MIN

#	Exercício	Tempo
1	Complete: o botão "Escolher da galeria"	7 min
2	Caça ao erro: o picker do tutorial de 2022	6 min
5	Associação: qual arquivo, qual endereço	4 min
6	Complete: o `_layout.tsx` com três telas	5 min
7	Caça ao erro: a navegação do tutorial	6 min

Para casa: 3 (seleção múltipla em `FlatList`) e 4 (câmera com os três estados — aparelho físico).

Hábito → `exercises.md` · Pet → `practice.md`

ATIVIDADE APLICADA — "O APP COM MAIS DE UMA TELA"

Migrar o projeto para o Expo Router e entregar **três rotas estáticas**:

- `/` — a lista que já existe desde a Aula 4
- **`/registrar-foto`** — galeria **ou** câmera, com prévia em `expo-image`
- **`/sobre`** — informações do app

Navegação por `Link` **e** por `router`.

O QUE SE AVALIA

1. Em `src/app/` **só** há telas e o `_layout.tsx`
2. O `_layout.tsx` dá **título a todas as telas**
3. `Link` onde o destino é fixo; `router` **depois** de uma ação — e o grupo sabe explicar
4. O picker trata **cancelar** e os **três estados** da câmera, e sempre oferece a galeria
5. **Nenhum zumbi:** `['images']`, `canceled`, `assets[0].uri`, nada de `@react-navigation/*`
6. O README **declara a limitação**: a foto escolhida não volta para a lista

HOJE VOCÊS TÊM TRÊS TELAS, E ELAS **NÃO CONVERSAM**.

A FOTO FICOU PRESA NA TELA ONDE FOI ESCOLHIDA. NA AULA 7, **AS TELAS COMEÇAM A CONVERSAR**.

ATALHOS DESTA APRESENTAÇÃO

Tecla	Ação
`→` / espaço · `←`	avançar · voltar
`S`	modo apresentador (notas + cronômetro)
`F`	tela cheia
`O` ou `Esc`	visão geral dos slides
`B` ou `.`	tela preta
`?`	todos os atalhos

Dados conferidos em 06/10/2026 · Expo SDK 57 · fontes em `lesson-plan.md` e `student-notes.md`